Project name: Vigenère Cryptography and Covering all Cases Exercise

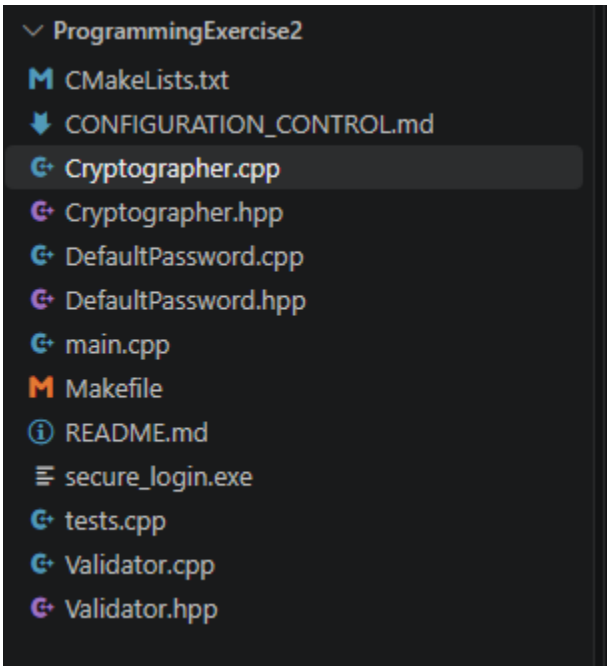
Module name: Login-v2 Module

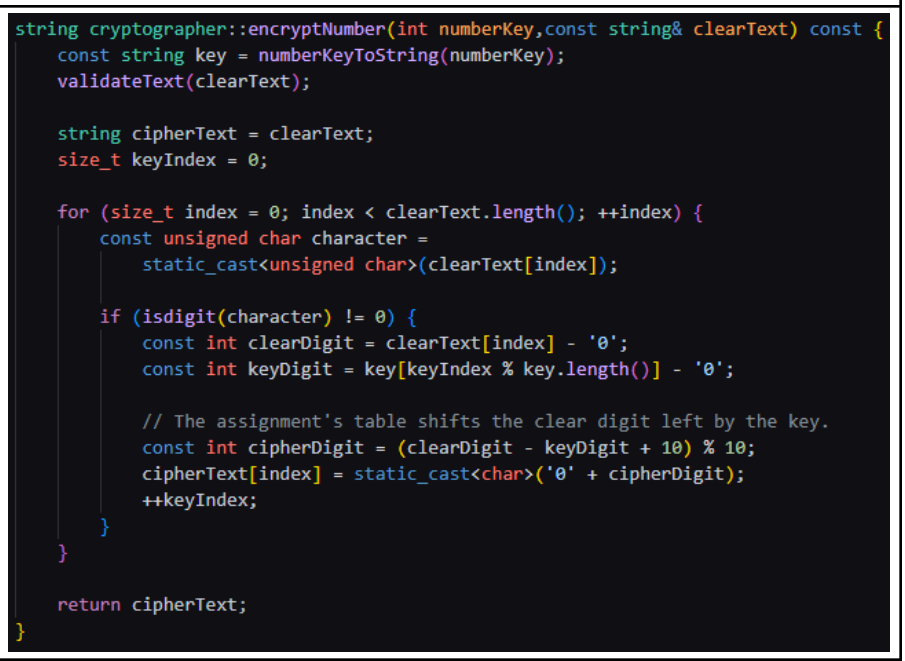
Developer: Jerome Ball

Course: CEN 4078 Secure Software Development

Date: 10/4/2026

Table 1. Login Module Test report

Data	***Proof*** - ***may be copy of code snippet*** - ***may be screen captures*** - ***may be written proof but must have data not just prose*** - ***check it after you submit. Make sure it is readable.*** - ***How about you make a table of inputs and results in addition to the above?***
Create a cryptographer class in your login module	 Cryptographer.cpp/hpp are the relevant files that contain the encryption/decryption logic

Create an encryption method based on the Vigenère algorithm	
Create a decryption method based on the Vigenère algorithm	
Create an encryption method based on the numerical Vigenère table	

```
string cryptographer::encryptVigenere(const string& alphaKey, const string& clearText) const {
    validateAlphaKey(alphaKey);
    validateText(clearText);

    string cipherText = clearText;
    size_t keyIndex = 0;

    for (size_t index = 0; index < clearText.length(); ++index) {
        const unsigned char character = static_cast<unsigned char>(clearText[index]);

        if (isalpha(character) != 0) {
            const unsigned char keyCharacter = static_cast<unsigned char>(alphaKey[keyIndex % alphaKey.length()]);
            const int shift = toupper(keyCharacter) - 'A';
            cipherText[index] = shiftLetter(clearText[index], shift);
            ++keyIndex;
        }
    }

    return cipherText;
}
```

```
string cryptographer::decryptVigenere(const string& alphaKey, const string& cipherText) const {
    validateAlphaKey(alphaKey);
    validateText(cipherText);

    string clearText = cipherText;
    size_t keyIndex = 0;

    for (size_t index = 0; index < cipherText.length(); ++index) {
        const unsigned char character =
            static_cast<unsigned char>(cipherText[index]);

        if (isalpha(character) != 0) {
            const unsigned char keyCharacter = static_cast<unsigned char>(alphaKey[keyIndex % alphaKey.length()]);
            const int shift = toupper(keyCharacter) - 'A';
            clearText[index] = shiftLetter(cipherText[index], -shift);
            ++keyIndex;
        }
    }

    return clearText;
}
```

```
string cryptographer::encryptNumber(int numberKey,const string& clearText) const {
    const string key = numberKeyToString(numberKey);
    validateText(clearText);

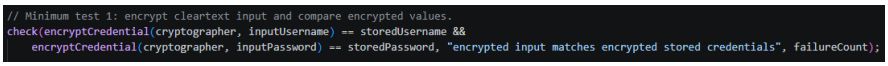
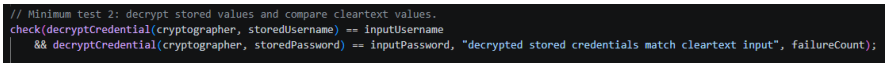
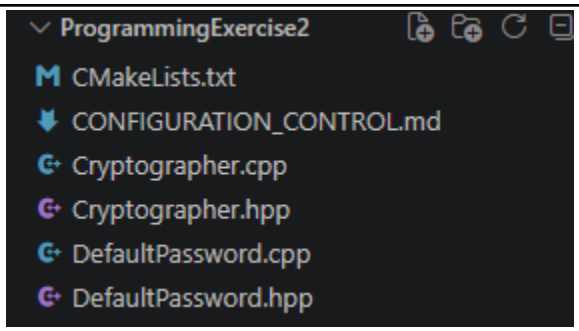
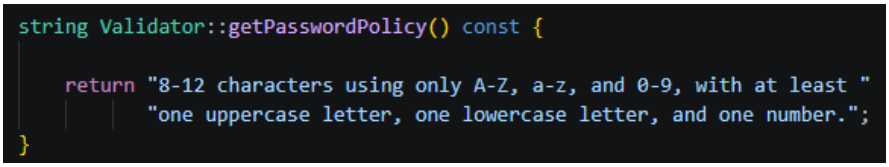
    string cipherText = clearText;
    size_t keyIndex = 0;

    for (size_t index = 0; index < clearText.length(); ++index) {
        const unsigned char character =
            static_cast<unsigned char>(clearText[index]);

        if (isdigit(character) != 0) {
            const int clearDigit = clearText[index] - '0';
            const int keyDigit = key[keyIndex % key.length()] - '0';

            // The assignment's table shifts the clear digit left by the key.
            const int cipherDigit = (clearDigit - keyDigit + 10) % 10;
            cipherText[index] = static_cast<char>('0' + cipherDigit);
            ++keyIndex;
        }
    }

    return cipherText;
}
```

Create an encryption method based on the numerical Vigenère table	
Validate the alphabetic and numerical Vigenère encryption	 Test function that checks if encryption is correct
Validate the alphabetic and numerical Vigenère decryption	 Test function that checks if decryption is correct
Create a default-password class in your login module	 The DefaultPassword.cpp/hpp are the relevant files
Create default-password method	DefaultPassword.cpp contains all logic to creating a default password.
Create a method which returns the password policy.	

<table>
<tr>
<td>Update your password validation code to user the method which returned the password policy.</td>
<td>

```
bool offerPasswordUpdate(User& user, array<User, 3>& users, const Validator& validator, const Cryptographer& cryptographer, const filesystem::path& databaseFile) {

    cout << "Create a new password for this account? (y/n): ";
    string answer;
    if (!getline(cin, answer)) {
        return false;
    }

    if (answer != "y" && answer != "Y") {
        return true;
    }

    cout << "Password policy: " << validator.getPasswordPolicy() << '\n';
    constexpr int passwordAttempts = 2;
```

Since we’re relying on pre-set accounts, we offer this if the user wants to change the password. Before allowing the user to change their password, the password policy is shown.
</td>
</tr>
<tr>
<td>Validate the method which creates the default-password.</td>
<td>

```
bool Validator::meetsPasswordPolicy(const string& password) const {
    if (password.length() < 8 || password.length() > 12) {
        return false;
    }

    const bool isAlphanumeric = all_of(password.begin(), password.end(), [](unsigned char character) { return isalnum(character) != 0; });
    const bool hasUppercase = any_of(password.begin(), password.end(), [](unsigned char character) { return isupper(character) != 0; });
    const bool hasLowercase = any_of(password.begin(), password.end(), [](unsigned char character) { return islower(character) != 0; });
    const bool hasNumber = any_of(password.begin(), password.end(), [](unsigned char character) { return isdigit(character) != 0; });

    return isAlphanumeric && hasUppercase && hasLowercase && hasNumber;
}
```
</td>
</tr>
<tr>
<td>Validate the method which returns the password policy.</td>
<td>

```
string Validator::getPasswordPolicy() const {

    return "8-12 characters using only A-Z, a-z, and 0-9, with at least "
           "one uppercase letter, one lowercase letter, and one number.";
}
```
</td>
</tr>
<tr>
<td>Validate the changes for testing passwords now that you must call the password policy.</td>
<td>

```
for (int attempt = 1; attempt <= passwordAttempts; ++attempt) {
    const string newPassword =
        readHiddenPassword("New password: ");

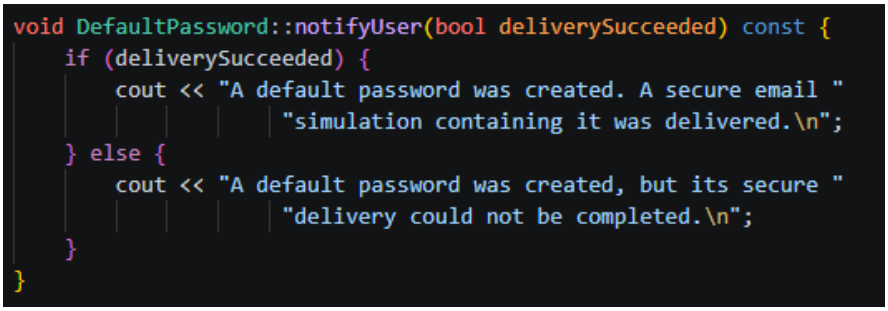
    if (validator.isSafeFromSqlInjection(newPassword) &&
        validator.meetsPasswordPolicy(newPassword)) {
        user.encryptedPassword =
            encryptCredential(cryptographer, newPassword);

        if (!saveCredentialDatabase(users, databaseFile)) {
            return false;
        }

        cout << "Password updated successfully.\n";
        return true;
    }

    cout << "Password creation failed. Please use the stated policy.\n";
}
```

Above code validates the new password
</td>
</tr>
</table>

Validate the notice to the user about the default password creation and notification.	
Did you meet all the requirements. If not, describe why?	Yes
Did you make changes to code from past assignments? If so, please describe including what requirements were tied to the code.	The only changes to existing code are to help implement the Vigenère cipher as well as the default password code.